

Query Evaluation

Internals of a DBMS

Query Optimization
And Execution

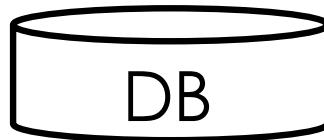
Relational Operators

Files and
Access Methods

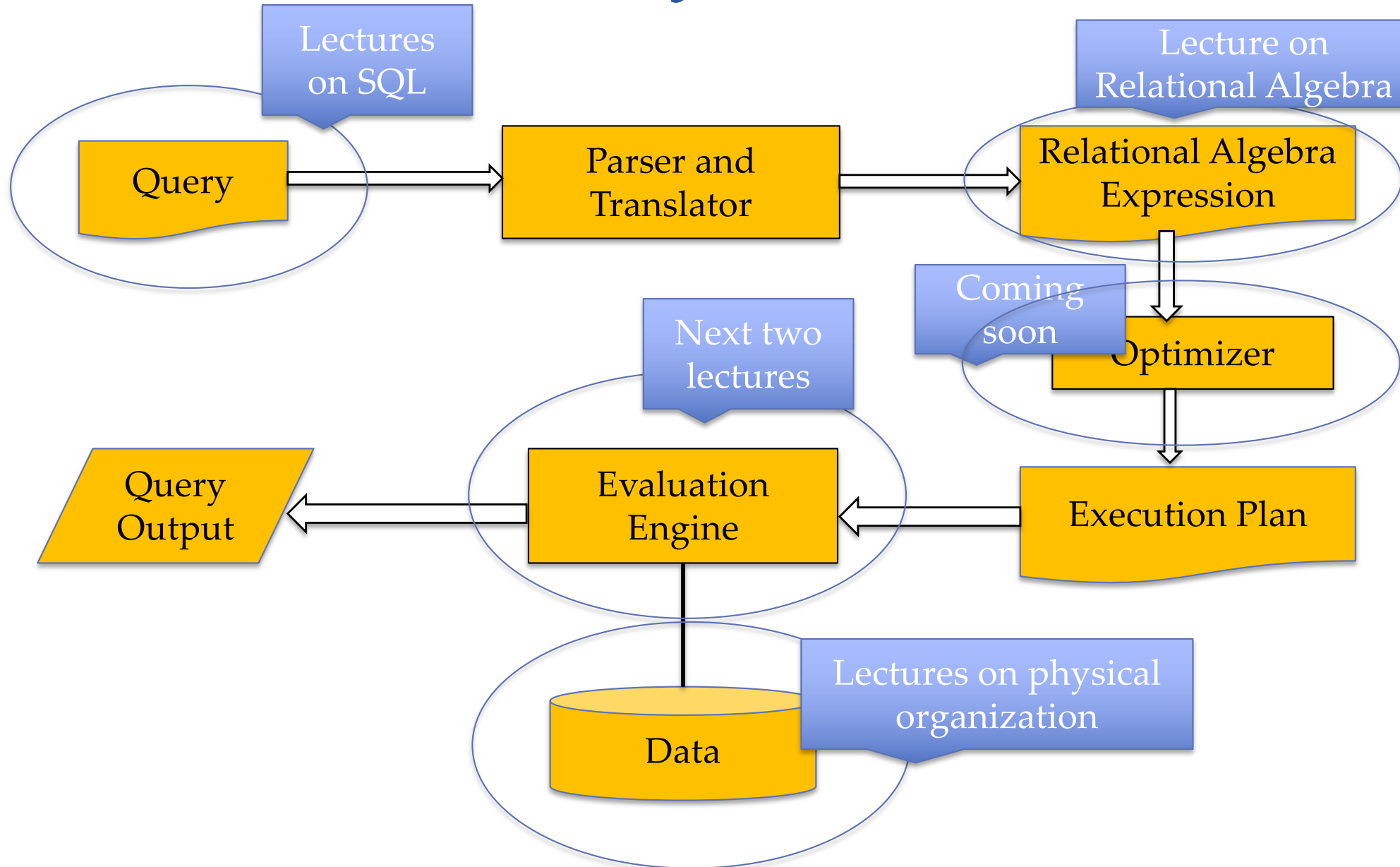
Buffer Management

Disk Space
Management

|



From Query to Result



Query Evaluation

- Processing of SQL queries
 - *Parser* translates into internal query representation
 - Based on relational algebra
 - The parser parses the string and checks for syntax errors
 - Checks existence of relations and attributes
 - Replaces views by their definitions
 - The *query optimizer* translates the query into an efficient execution plan
 - Many different execution plans exist;
 - Choosing an efficient execution plan is crucial
 - The *plan executor* executes the execution plan and delivers data

Processing Individual Operators

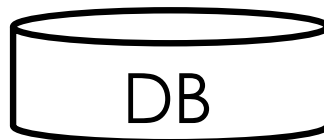
Query Optimization
And Execution

Relational Operators

Files and
Access Methods

Buffer Management

Disk Space
Management



Let's start with the individual operators

- Query decomposition
 - Queries are composed of **few basic operators**
 - Selection
 - Projection
 - Order by
 - Join
 - Group by
 - Intersection
 - ...
 - Several alternative algorithms for implementing each relational operator
 - Not a single “best” algorithm; efficiency of each implementation depends on factors like size of the relation, existing indexes, size of buffer pool etc.

Cost model and notation

- In order to compare different alternatives to implement an operator we must estimate how expensive a specific execution is
- Input for cost model:
 - the query and database statistics (e.g., distribution of values, etc.)
 - The number of available memory blocks (buffer pool frames available to use for the query): B
 - ... other details about resources that we ignore
- Our cost metric
 - Number of I/O = pages retrieved from disk
 - assumption that the root and all intermediate nodes of the B+ are in main memory:
 - leaf pages and data pages may not be in main memory!!!
 - A page P might be retrieved several times if many pages are accessed between two accesses of P
 - (Main Memory Requirement)

Schema for Examples

Users (uid: int, *uname*: string, *experience*: int, *age*: int)

GroupMembers (uid: int, gid: int, *stars*: int)

- **Users (U):**

- 40,000 tuples (denoted as |U| or CARD(U))
- Around 80 tuples per data page/block
- 500 data pages/block (denoted as UserPages)
- An index on uid has around 170 leaf pages
- An index on uname has around 300 leaf pages

1, Billy, 5, 30
5, Dora, 9, 25
...
88, Ann, 2, 12

- **GroupMembers (GM):**

- 100,000 tuples (denoted as |GM| or CARD(GM))
- Around 100 tuples per data page/block
- Total of 1000 data pages (denoted as GroupMemberPages)

- **Database statistics tables**

- Keep track of cardinality of tables, number of pages/blocks, what indexes, how big an index, etc.
- keep track of domain of values

8 • E.g., experience values: 1.....10

Selections

```
SELECT  *  
FROM    Users  
WHERE   uid = 123
```

```
SELECT  *  
FROM    Users  
WHERE   uname = 'Billy'
```

```
SELECT  *  
FROM    Users  
WHERE   uname < 'G'
```

- General form: $\sigma_{R.attr \ op \ value} (R)$
- Execution Alternatives:
 - scan the entire relation (can be done whether an index exists or not)
 - use an index on the condition attribute (if such index exist)

Selection

Selectivity / Reduction Factor

- Given a relation R, the **Reduction Factor** of a condition on R is defined as
 - $\text{Red}(\sigma_{\text{condition}}(R)) = |\sigma_{\text{condition}}(R)| / |R|$
- Example
 - $\text{Red}(\sigma_{\text{experience}=5}(\text{Users})) = |\sigma_{\text{experience}=5}(\text{Users})| / |\text{Users}| = ?$
 - Assume we know that 10,000 users have experience of 5
 - $10.000/40.000 = 0.25$

Selection

Selectivity / Reduction Factor

How to generally calculate Reduction factor if $|\sigma_{\text{condition}}(R)|$ not known

- Most common: DBMS keeps track of number of different values and assumes uniform distribution
- Reduction Factor = Size of selected range / total size of domain
- 10 different experience levels for User
 - $\text{Red}(\sigma_{\text{experience}=5}(U)) = 1/|\text{different experience levels}| = 0.1$
- Users between 11 (min age) and 60 (max age) years old
 - $\text{Red}(\sigma_{\text{age} \leq 15}(U)) = (15-10) / (60-10) = 5/50 = 0.1$
- $\text{Red}(\sigma_{\text{experience}=5 \text{ and age } \leq 15}(U)) = ?$
 - Assume uniform and independent distribution
 - $\text{Red}(\sigma_{\text{experience}=5}(U)) * \text{Red}(\sigma_{\text{age} \leq 15}(U))$

Selection

Selectivity / Reduction Factor

- Result sizes: number of input tuples multiplied by reduction factor
 - Assume reduction factor of a condition is 0.1
 - Assume 40000 tuples
 - Result size = $40000 * 0.1 = 4000$ tuples
- How does the DBMS know number of different values, how many of a certain value, max, min...?
 - through indices, heuristics, separate statistics (histograms)

Simple Selections

```
SELECT  *  
FROM    Users  
WHERE   uid = 123
```

Assumptions:

- Uid is primary key

```
SELECT  *  
FROM    Users  
WHERE   uname = 'Billy'
```

- 100 records with uname 'Billy'

```
SELECT  *  
FROM    Users  
WHERE   uname < 'G'
```

- 10,000 records with uname A-F

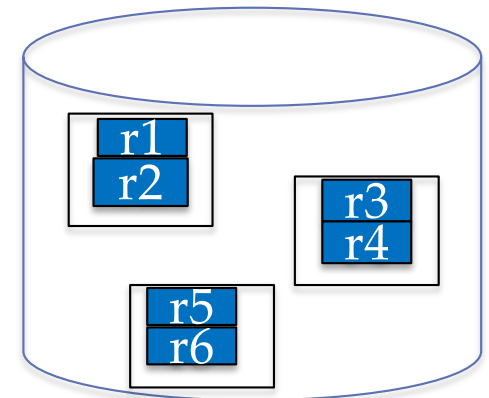
Operator alternatives for selections

```
SELECT  *  
FROM    Users  
WHERE   uid = 123
```

```
SELECT  *  
FROM    Users  
WHERE   uname = 'Billy'
```

```
SELECT  *  
FROM    Users  
WHERE   uname < 'G'
```

- Simple scan on data pages:
 - Always possible; even if there are indices
 - Search on arbitrary attribute(s): scan the entire table
 - I/O: UserPages=500
 - Search on primary key attribute:
 - scan on average half of U (cost is UserPages/2=250)



Operator alternatives simple selections

```
SELECT  *  
FROM    Users  
WHERE   uid = 123
```

```
SELECT  *  
FROM    Users  
WHERE   uname = 'Billy'
```

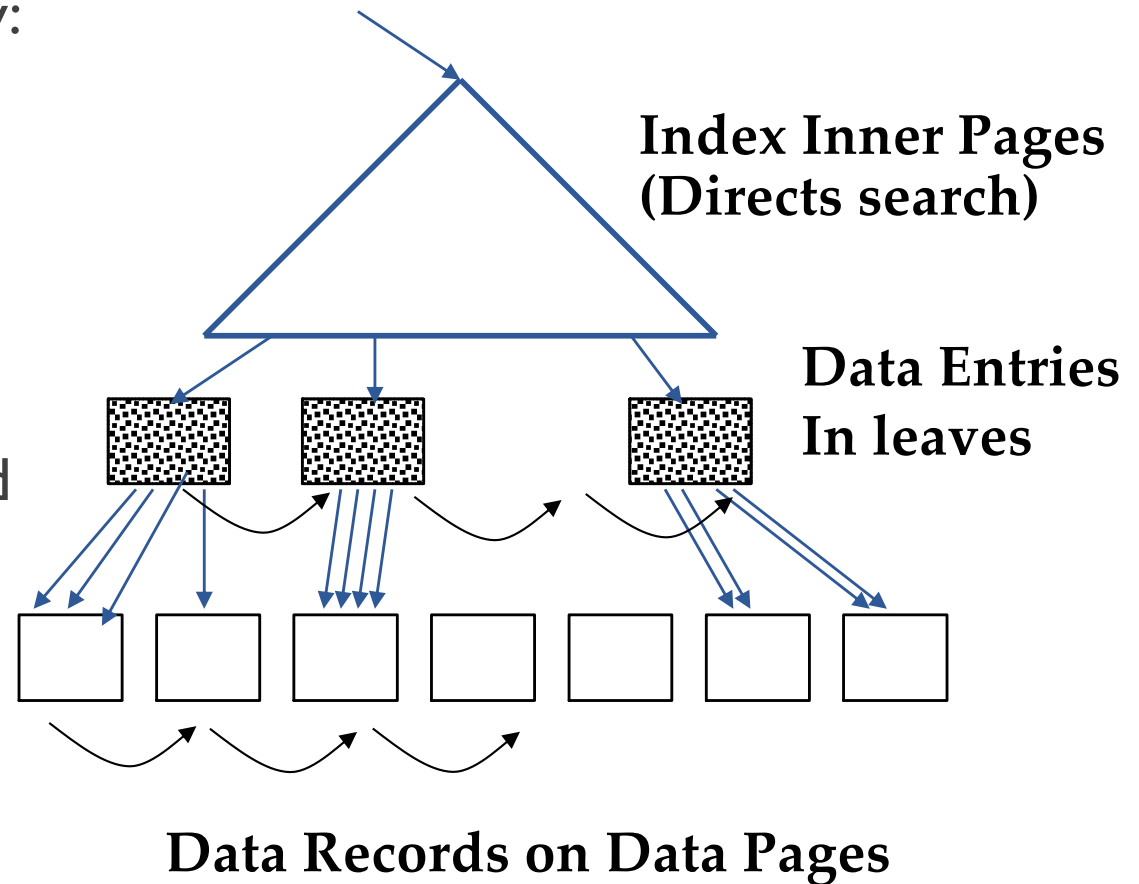
```
SELECT  *  
FROM    Users  
WHERE   uname < 'G'
```

- Index on selection attributes:
 - Use index to find qualifying data entries, then retrieve corresponding data records.
 - I/O:
 - No I/O for root access or access to intermediate nodes
 - I/O for certain number of leaf pages that contain matches
 - I/O to retrieve data pages that have matching tuples
 - Now how much is that??
 - Depends on number of qualifying tuples
 - Depends on type of index

In case of Clustered B+ Tree

Cost:

- Path from root to the left-most leaf **lq** with qualifying data entry:
 - Inner pages in memory
 - One I/O to retrieve **lq** page
- Retrieve page of first qualifying tuple: 1 I/O
- Retrieve all following pages as long as search criteria is fulfilled
- Each data page only retrieved once
- # data pages
 - #matching tuples / tuples per page
 - E.g., if 20% of tuples qualify then roughly 20% of data pages are retrieved



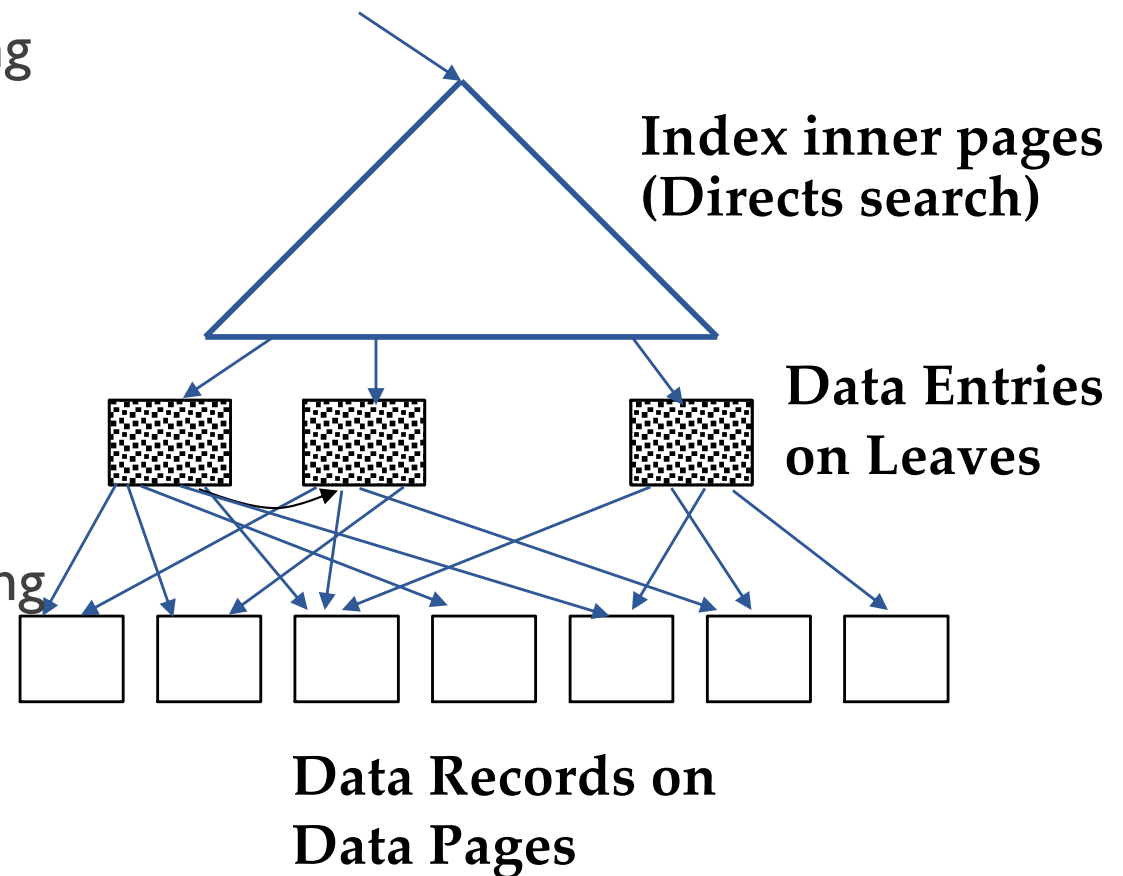
Clustered Index for our Example

- Clustered: matching tuples are in adjacent data pages:
 - # datapages = number of matching tuples / number of tuples per page
 - Example 1: **SELECT * FROM Users WHERE uid = 123**
 - 1 tuple matches because primary key (1 data page)
 - I/O cost: 1 index leaf page + 1 data page
 - Example 2: **SELECT * FROM Users WHERE uname = 'Billy'**
 - We assumed 100 matching tuples
 - Since clustered they are on adjacent 2(-3) pages
 - I/O cost: 1 index leaf page + two data pages
 - Example 3: **SELECT * FROM Users WHERE uname < 'G'**
 - Estimate is that around 10000 tuples match
 - i.e., if 25% of the data, then clustered on approx. 25% of the pages (125 data pages)
 - Cost: 1 leaf pages + 125 data pages
 - Note: some systems might retrieve all rids through the index (not efficient).
 - In this case, example 3 will read approx. 25% of the leaf pages

In case of non-clustered B+ Tree

Cost:

- Path to first qualifying leaf:
 - One I/O to retrieve **lq** page
- Retrieve page of first qualifying tuple: 1 I/O
- Retrieve page of second qualifying tuple
- ...
- Retrieve next leaf page with qualifying tuple
- Retrieve page of next qualifying tuple
- Sometimes page might have been retrieved previously
 - Might still be in main memory
 - Might have been replaced again



Use non-clustered index: simple strategy

- Standard strategy:
 - get one page after the other:
 - Worst case #data pages = #matching tuples
 - Example 1: **SELECT * FROM Users WHERE uid = 123**
 - Same as before
 - Example 2: **SELECT * FROM Users WHERE uname = 'Billy'**
 - Estimated that around 100 tuples match
 - Index: The data entry for that will be on 1 leaf page of index (or two if an entry can span more than one page)
 - Data pages: In worst case each of the 100 tuples on a different data page
 - But even if two are on the same page, when the second record is retrieved the page might already be no more in main memory..
 - cost: 1-2 index-leaf-page + 100 pages (some pages might be retrieved several times)

Use non-clustered index: simple strategy

- Standard strategy continued:
 - Example 3: **SELECT * FROM Users WHERE uname < 'G'**
 - Estimated that 10000 tuples match = 25%
 - Likely that nearly every data page has a matching tuple!
 - But as we retrieve tuple by tuple, every retrieval might lead to I/O as page might have been purged from main memory in between
 - cost: 75 leaf pages + 10000 pages (most pages will be retrieved several times)
 - 75 leaves = 25% of all leaf pages
 - Simple scan is faster!!
 - Lesson learned:
 - Non-clustered Index usually only useful with very small reduction factors

Use non-clustered Index with Sorting

- Example 2: **SELECT * FROM Users WHERE uname = 'Billy'**
 - Example 3: **SELECT * FROM Users WHERE uname < 'F'**
 - Determine all leaf pages that have matching entries (1-2 or 75 leaf pages)
 - Assume that all 2 or 75 leaf pages fit into main memory
 - sort matching data entries (rid=pid,slot-id) in leaf-pages by page-id
 - Your favorite main memory sorting algorithm
 - Retrieve each page only once and get all matching tuples
 - #data pages = #data pages that have at least one matching tuple;
 - Example 2: worst case is total is 100 but likely less
 - Example 3: worst case is # of data pages
 - For Example 3
 - Around 10000 tuples match
 - If they are distributed over 490
 - cost: 75 leaf pages + 490 pages (worse than a scan)
 - If they are distributed over 300 pages
 - Cost 75 leaf pages + 300 pages (better than a scan)
- 21 Note: sorting expensive if leaf-pages do not fit in main-memory

Selections on 2 or more attributes

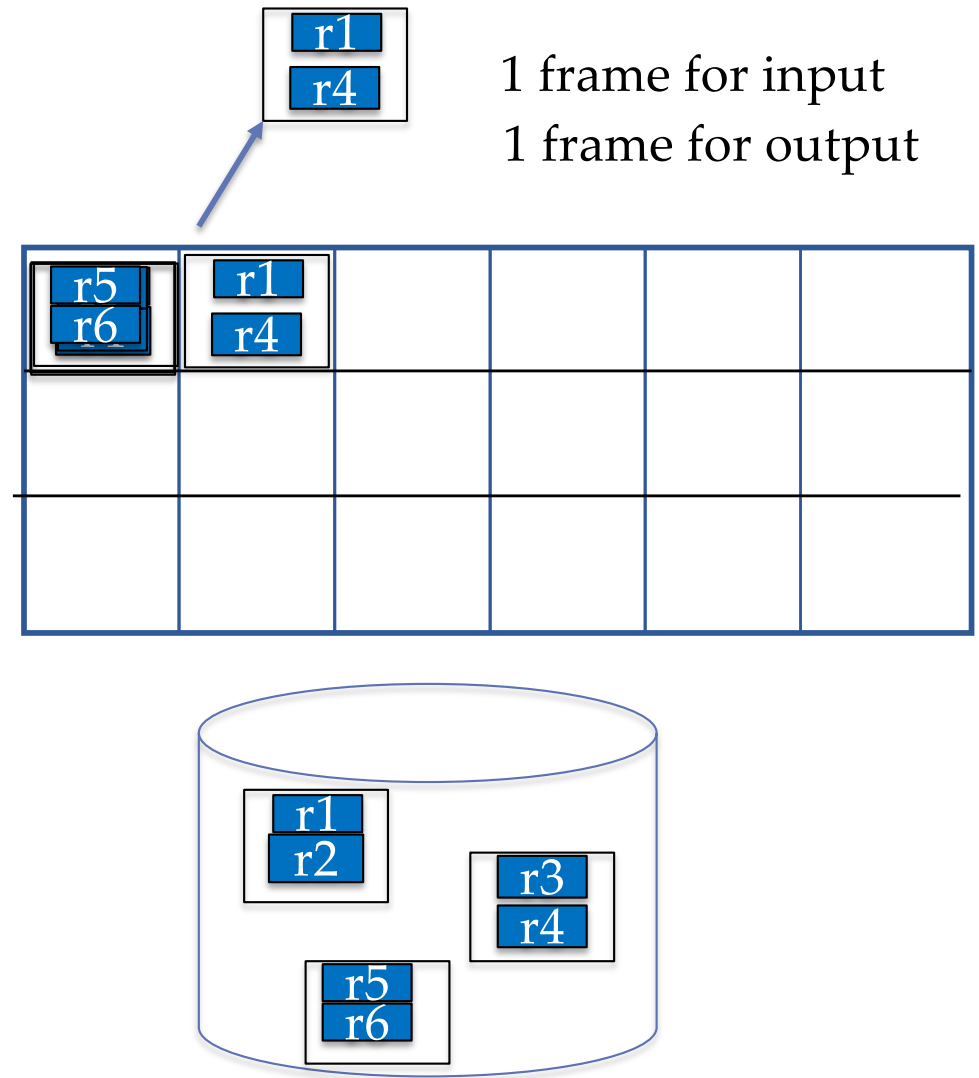
- $A=100 \text{ AND } B=50$ (pages = 500)
 - No index:
 - 500
 - Index on A attribute:
 - Get all tuples for $A=100$ through index, check value of B
 - Cost same as the query for $A=100$
 - Index on B attribute: similar to index on A
 - 2 indexes; one for each attribute
 - Intersection based:
 - Find rids where $A = 100$ through A index
 - Find rids where $B = 50$ through B index
 - Build intersection of rids
 - Retrieve from data pages all tuples with rids in that intersection
 - 1 index with both attributes
- $A=100 \text{ and } B<50$ ($\text{Red}(B<50) = 0.5$)
 - Very low reduction factor for $B<50$, $\rightarrow$ index on B not much use

Selections on 2 or more attributes

- $A=100 \text{ OR } B=50$ (pages = 500)
 - No index:
 - 500
 - Index on A attribute alone or index on B attribute alone
 - Not useful
 - 2 indexes; one for each attribute
 - Find rids where $A = 100$ through A index
 - Find rids where $B = 50$ through B index
 - Build union of rids
 - Retrieve from data pages all tuples with rids in that union
 - 1 index with both attributes (A,B)
 - Not much use

main memory usage: Scan

- A scan of relation only needs two main memory frames
 - So works the same if you have large or small buffer



main memory usage: index

- Index standard additionally

- Must hold root and intermediate pages
- Must have space to retrieve one leaf page at a time

- Index with sorting / intersection...

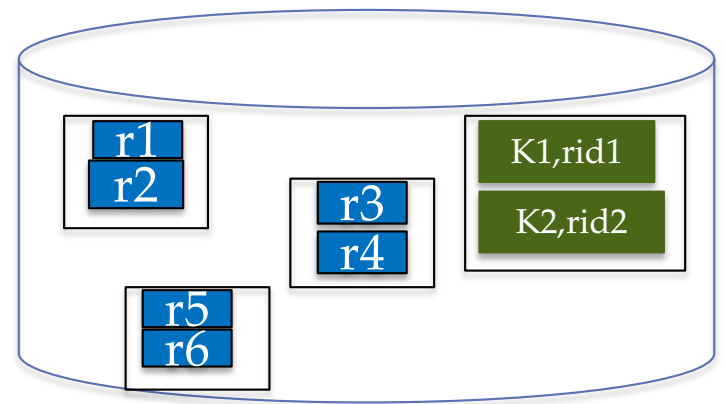
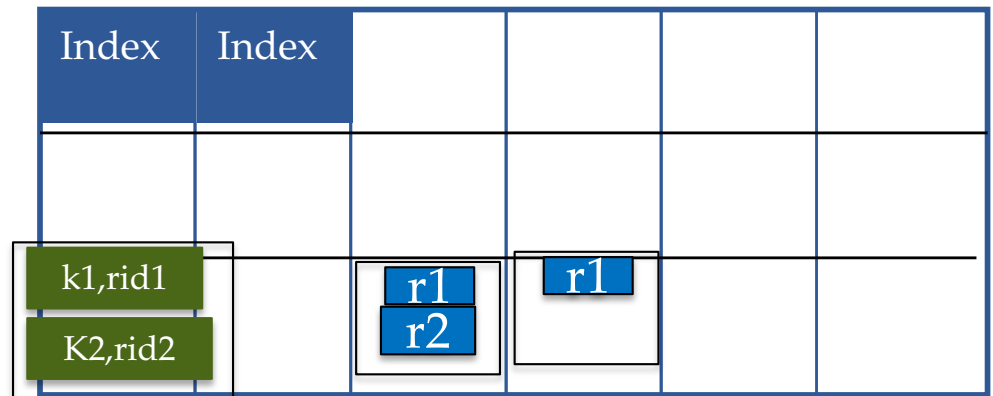
additionally

- Must hold root and intermediate pages
- Must hold necessary leaf pages for sort / intersection etc.

Frames for inner pages
(we generally ignore)

1 frame for 1 leaf page
1 frame for 1 data page

1 frame for output



The Projection Operation

```
SELECT  uid, experience
FROM    Users
```

- Scan relation, read each tuple, extract relevant attributes only and return to user
- Projection is often together with other operators in one query

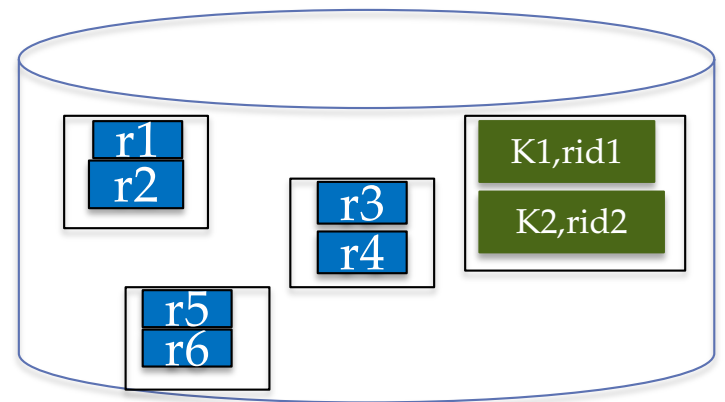
```
SELECT  uid, experience
FROM    Users
WHERE   uid = 123
```

- Decide how to do the other operation and do the projection on the fly
 - For instance: use index for selection, get tuple and then extract relevant attributes before pipelining result to client.

With index usage

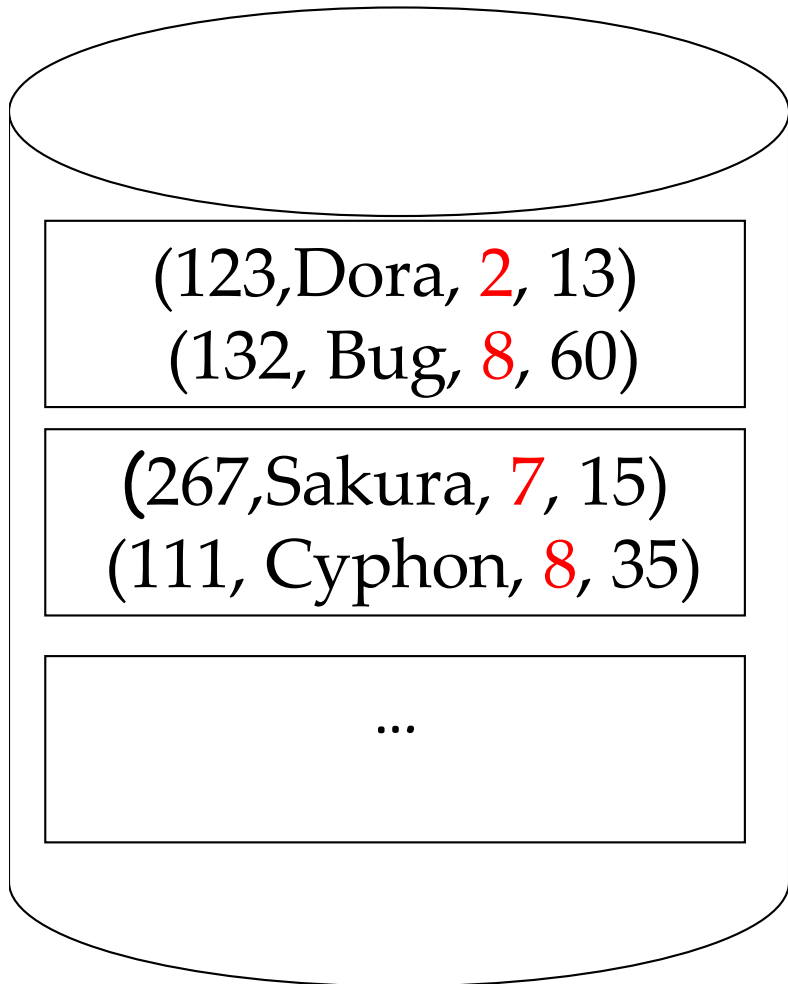
- When creating the output tuple
 - Only create tuple with relevant attributes

Index	Index				
k1,rid1		r1			
K2,rid2		r2			

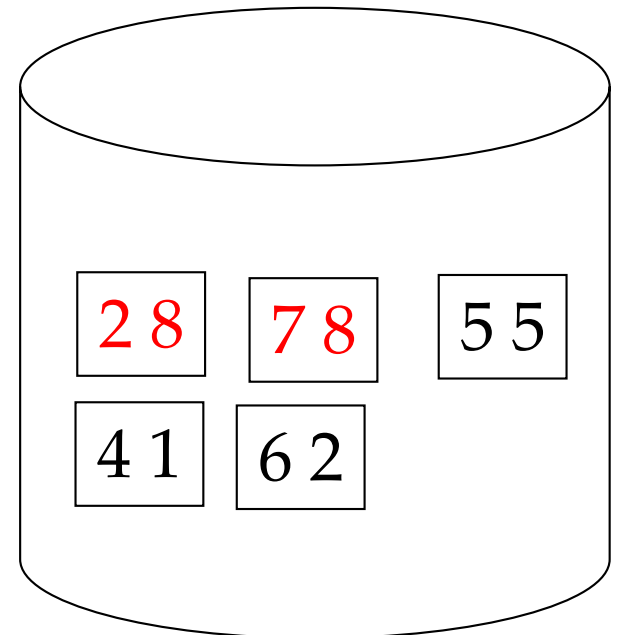


External Sorting

Example Setup



Represented in the following as:



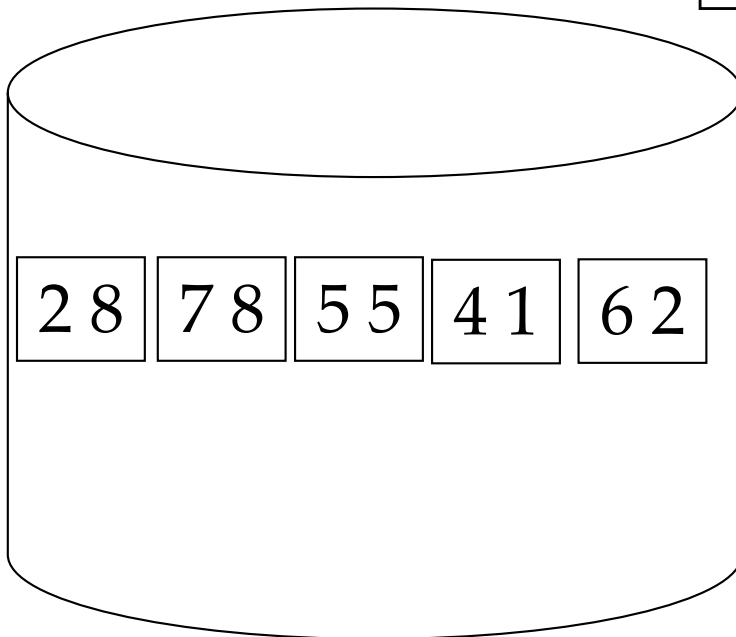
Summary of example:

- 5 pages
- Each with two tuples

External Sorting

Task:

```
SELECT  *  
FROM    Users  
ORDER BY experience
```

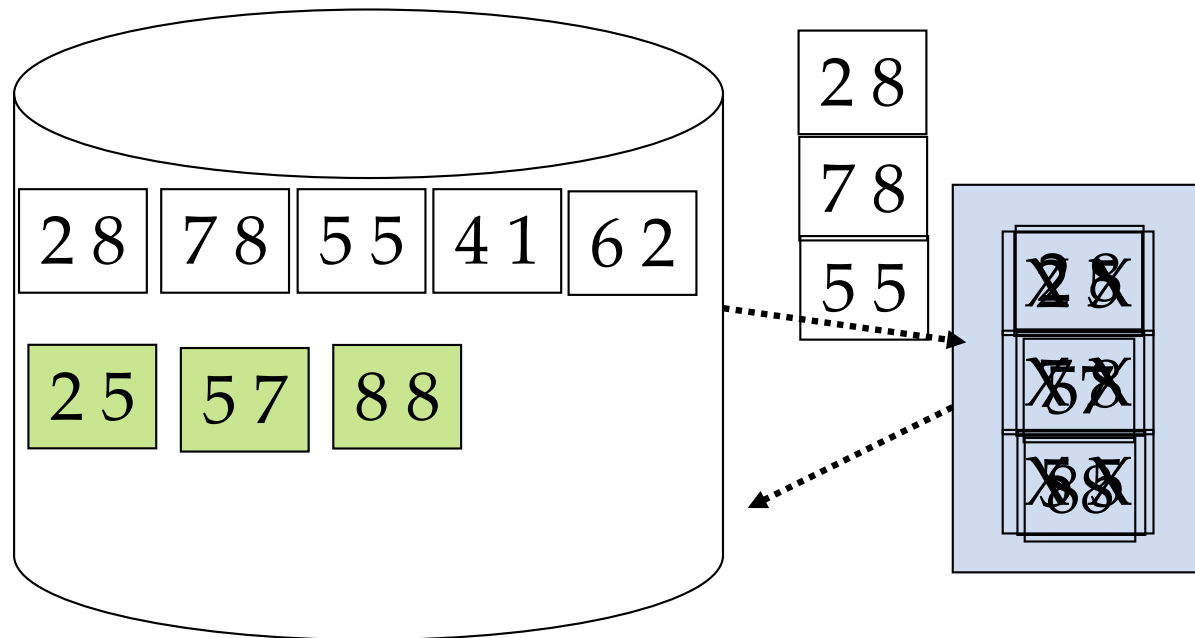


Summary of example:

- 5 pages
- Each with two tuples

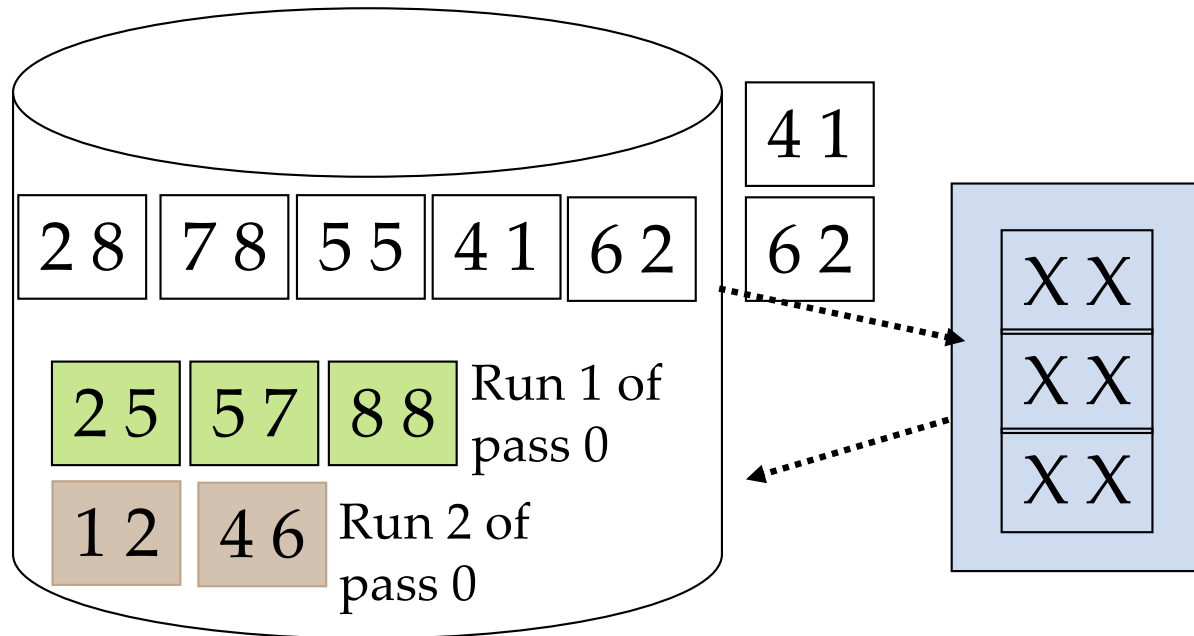
What if only 3 buffer frames available?
That is, data pages do not
fit into main memory

External Sorting - Pass 0



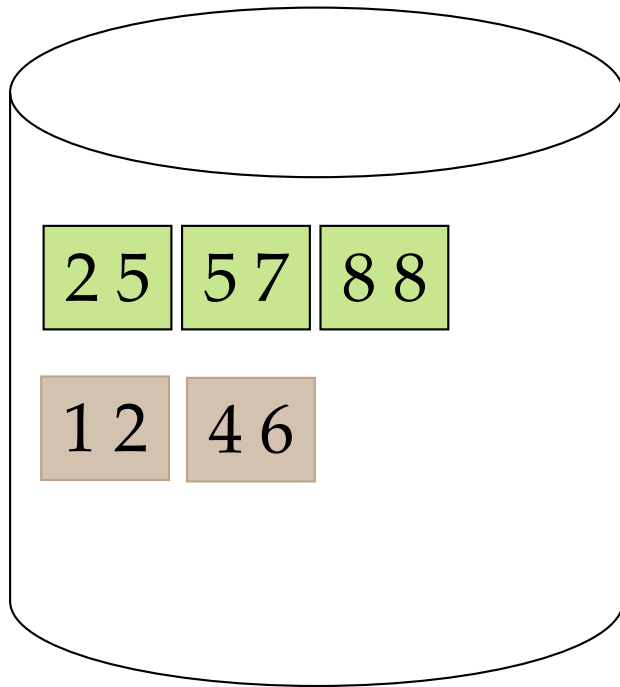
- In example: 5 pages, 3 buffer frames
- N pages, B buffer frames:
 - Bring B pages in buffer
 - Sort with any main memory sort
 - Write out to disk to a temporary file;
 - it's called a run of B pages

External Sorting - Pass 0

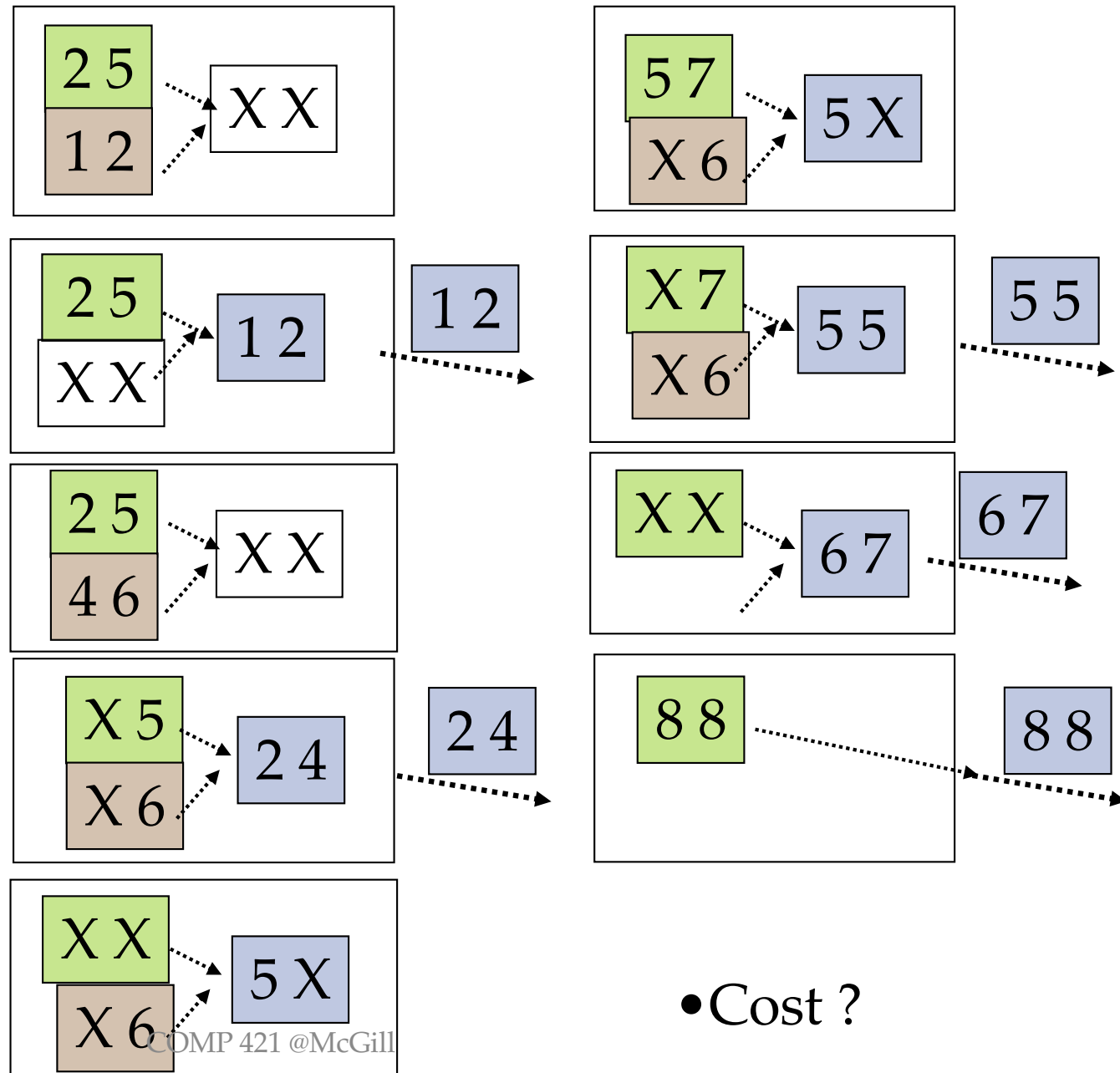


- In example: $5/3 = 2$ runs (round up!)
- In total N/B runs
- Cost ?

External Sorting Pass 1:



- N/B input frames
- One output frame
- Merge Sort the runs of pass 0
- Hopefully $B-1 > N/B$
- Otherwise pass 2...



Sort

- Sometimes a Pass 2 is needed:
 - Pass 0 created more runs than there are main memory buffers
 - Therefore Pass 1 produces more than one run
 - Take the first $B-1$ runs from pass 0 and merge them to one bigger Pass 1 run
 - Then take the next $B-1$ runs from pass 0 and merge them to one bigger Pass 1 run
 - ...
 - Pass 2 takes the runs of Pass 1 and merges them
 - If there are less than B Pass 1 runs, then this is the final pass
 - Otherwise Pass 3...
- Number of passes: $1 + \lceil \log_{B-1} \lceil N / B \rceil \rceil$

Typically no write I/O for last pass

- I/O Costs:
 - `SELECT * FROM Users ORDER BY experience`
 - If everything fits into main memory (Only pass 0 needed):
 - Read number of data pages
 - sort and pipeline output pages directly to client
 - Pass 0 + pass 1 needed
 - Pass 0: read # pages, write # pages (have to write temp. results!)
 - Pass 1: read # pages, sort and pipeline result to client
 - $3 * \text{\#pages}$
 - Pass 0 + pass 1 + pass 2 needed
 - $5 * \text{\#pages}$
 - $2N * \text{number of passes} - N$
- Memory requirement:
 - minimum $B = 3$
- the bigger B , the better!!

Sort Costs

Number of passes

N	B=3	B=5	B=9	B=17	B=129	B=257
100	7	4	3	2	1	1
1,000	10	5	4	3	2	2
10,000	13	7	5	4	2	2
100,000	17	9	6	5	3	3
1,000,000	20	10	7	5	3	3
10,000,000	23	12	8	6	4	3
100,000,000	26	14	9	7	4	4
1,000,000,000	30	15	10	8	5	4